ASSIGNMENT 5.2

Capstone: Consolidated Autonomous Placement Agent

Student	[Student Name]	Roll No	[Roll No]
Course	21CSE525T - Agent Technology	Project	Placement Preparation Agent

Integration of Units 1 to 5 in one running system. File: assignments/a5_2/capstone_agent.py

1. Objective

Combine every earlier unit into one agent that runs its own cycles, senses its environment, negotiates, plans and defends itself, with no human input during the run.

2. How the units fit together

- **Unit 1 (reflex):** ReflexLayer applies condition-action rules to each percept.
- **Unit 2 (daemons and bus):** CodingDaemon, AptitudeDaemon, DriveDaemon send signed percepts onto a shared queue; the main loop drains it.
- **Unit 3 (negotiation and utility):** UtilityOptimizer and NegotiationModule choose a mock-interview contract (Contract Net).
- **Unit 4 (BDI and knowledge):** BDICore holds beliefs, desires and intentions; KnowledgeGraph provides prerequisite and eligibility inference.
- **Unit 5 (security):** SecurityGateway verifies every incoming message before it can change belief.
- **Dashboard:** a live text dashboard updates each cycle.

3. Autonomous cycle

Each cycle: sense (drain the bus, verify signatures, revise beliefs), reflex pass, deliberate (choose the active desire), execute one intention step, and render the dashboard.

An IntruderDaemon sends unauthenticated and malformed messages throughout the run, so the security layer is tested under live load.

4. Implementation

```
class SecurityGateway:
    def __init__(self):
        self.nonces, self.ok, self.blocked = set(), 0, 0
        self.blocked_events = []

    def verify(self, m):
        b, s = m.get("body", {}), m.get("sig", "")
        if not all(k in b for k in ("host_id", "command", "args", "nonce", "ts")):
            return self._no("malformed payload")
        h = TRUSTED.get(b["host_id"])
        if not h:
            return self._no(f"untrusted host '{b['host_id']}'")
        exp = hmac.new(h["secret"], canon(b), hashlib.sha256).hexdigest()
        if not hmac.compare_digest(exp, s):
            return self._no(f"signature mismatch from '{b['host_id']}'")
        if b["nonce"] in self.nonces:
            return self._no("replay attack (nonce reuse)")
        if abs(int(time.time()) - b["ts"]) &gt; 60:
            return self._no("stale message")
        if b["command"] not in PERMS[h["role"]]:
            return self._no(f"privilege escalation: {h['role']} -&gt; {b['command']}")
        self.nonces.add(b["nonce"])
        self.ok += 1
        return b

    def _no(self, reason):
        self.blocked += 1
```

```
        self.blocked_events.append(reason)
        log(5, "SECURITY", f"BLOCKED: {reason}", C_ERR)
        return None

class BDICore:
    def __init__(self, kg):
        self.kg = kg
        self.B = BeliefBase()
        self.D: List[Desire] = []
        self.I: List[Intention] = []
        self.dropped: List[Intention] = []
        self._init()

    def _init(self):
        for k, v in {"cgpa": 7.6, "days_to_drive": 40, "aptitude": 62,
                     "interview_rating": 5, "target": "Freshworks",
                     "levels": {"arrays": 81, "strings": 76, "oops": 72, "recursion": 58,
                                "dp": 39, "graphs": 44, "trees": 63, "dbms": 68,
                                "os": 55, "aptitude": 62}}.items():
            self.B.set(k, v)

        self.D = [
            Desire("SURVIVE_DRIVE", 10.0,
                   lambda b: b.get("days_to_drive") &lt;= 7,
                   lambda b: b.get("days_to_drive") &lt; 0,
                   lambda b: 3.0 if b.get("days_to_drive") &lt;= 3 else 2.0),
            Desire("FIX_WEAK_SKILL", 8.0,
                   lambda b: len(self.kg.gaps(b.get("levels"), b.get("target"))) &gt; 0,
                   lambda b: len(self.kg.gaps(b.get("levels"), b.get("target"))) == 0),
            Desire("PASS_APTITUDE", 7.0,
                   lambda b: b.get("aptitude") &lt; 75, lambda b: b.get("aptitude") &gt;= 75),
            Desire("INTERVIEW_READY", 6.5,
                   lambda b: b.get("interview_rating") &lt; 8,
                   lambda b: b.get("interview_rating") &gt;= 8),
        ]

    def deliberate(self):
        active = []
        for d in self.D:
            if d.sat(self.B):
                if d.status != DStatus.ACHIEVED:
                    d.status = DStatus.ACHIEVED
                    log(4, "DESIRE", f"'{d.name}' ACHIEVED", C_OK)
                continue
            if d.trig(self.B):
                d.status = DStatus.ACTIVE
                active.append(d)
            else:
                d.status = DStatus.INACTIVE
        active.sort(key=lambda d: -d.prio(self.B))
        if not active:
            return None
        top = active[0]
        cur = self.I[-1] if self.I else None
        if cur is None:
            p, s = PLANS[top.desire if hasattr(top, 'desire') else top.name]
            self.I.append(Intention(top.name, p, list(s)))
            log(4, "INTENTION", f"COMMIT {p} (desire {top.name}, prio {top.prio(self.B)})")
        elif cur.desire != top.name:
            cd = next(d for d in self.D if d.name == cur.desire)
            if top.prio(self.B) &gt; cd.prio(self.B):
                log(4, "RECALC", f"priority inversion {top.name}({top.prio(self.B)}) &gt; "
                                 f"{cur.desire}({cd.prio(self.B)})", C_WARN)
                self.dropped.append(self.I.pop())
                log(4, "INTENTION", f"DROP {cur.plan} at {cur.prog()}", C_WARN)
                p, s = PLANS[top.name]
                self.I.append(Intention(top.name, p, list(s)))
                log(4, "INTENTION", f"COMMIT {p} (re-planned)", C_OK)
        return self.I[-1] if self.I else None

    def execute(self):
        if not self.I:
            return None
        cur = self.I[-1]
        if cur.done():
            log(4, "INTENTION", f"COMPLETE {cur.plan}", C_OK)
            self.I.pop()
            return "COMPLETE"
        s = cur.step()
        log(4, "EXECUTE", f"[{cur.plan} {cur.prog()}] {s}")
```

```
        return s

class PlacementPreparationAgent:
    def __init__(self):
        self.kg = KnowledgeGraph()
        self.bdi = BDICore(self.kg)
        self.sec = SecurityGateway()
        self.reflex = ReflexLayer()
        self.opt = UtilityOptimizer()
        self.neg = NegotiationModule()
        self.percepts = 0
        self.reflex_fires = 0
        self.cycle = 0
        self.contract = None
        self.selected_resource = None

    # ---------------- SENSE (Unit 1 + 2 + 5) -------------------------
    def sense(self):
        drained = []
        while True:
            try:
                drained.append(EVENT_BUS.get_nowait())
            except queue.Empty:
                break
        for m in drained:
            body = self.sec.verify(m)      # UNIT 5 gate
            if not body:
                continue
            self.percepts += 1
            a = body["args"]
            with STATE_LOCK:
                if body["command"] == "PERCEPT" and a["type"] == "CODING":
                    lv = dict(self.bdi.B.get("levels"))
                    lv[a["skill"]] = max(0, min(100, lv.get(a["skill"], 50) + a["delta"]))
                    self.bdi.B.revise("levels", lv, f"coding-daemon/{a['skill']}")
                elif body["command"] == "PERCEPT" and a["type"] == "APTITUDE":
                    self.bdi.B.revise("aptitude", a["score"], "aptitude-daemon")
                    lv = dict(self.bdi.B.get("levels")); lv["aptitude"] = a["score"]
                    self.bdi.B.set("levels", lv)
                elif body["command"] == "PUSH_DRIVE":
                    self.bdi.B.revise("days_to_drive", a["days"],
                                      f"placement-cell/{a['company']}")
                    self.bdi.B.revise("announced_company", a["company"], "placement-cell")
        return len(drained)

    # ---------------- REFLEX (Unit 1) --------------------------------
    def reflex_pass(self):
        lv = self.bdi.B.get("levels")
        weakest = min(lv, key=lv.get)
        s = {"days": self.bdi.B.get("days_to_drive"), "apt": self.bdi.B.get("aptitude"),
             "mock": self.bdi.B.get("interview_rating"), "hours": round(RNG.uniform(0, 5), 1),
             "weakest": weakest, "weakest_lvl": lv[weakest]}
        f = self.reflex.react(s)
        self.reflex_fires += len(f)
        return f

    # ---------------- DASHBOARD (UI layer) ---------------------------
    def dashboard(self):
        B = self.bdi.B
        lv = B.get("levels")
        print("\n" + "+" + "-" * 86 + "+")
        print(f"| {'PLACEMENT PREPARATION AGENT :: LIVE DASHBOARD':&lt;60}"
              f"cycle {self.cycle:&gt;3}  {now()}  |")
        print("+" + "-" * 86 + "+")
        print(f"| CGPA {B.get('cgpa')}   Target: {B.get('target'):&lt;12} "
              f"Drive in: {B.get('days_to_drive')}d   "
              f"Aptitude: {B.get('aptitude')}%   Mock: {B.get('interview_rating')}/10"
              f"{'':&gt;4}|")
        print("+" + "-" * 86 + "+")
        bar = ""
        for k, v in sorted(lv.items(), key=lambda x: x[1]):
            filled = int(v / 10)
            col = C_ERR if v &lt; 50 else (C_WARN if v &lt; 70 else C_OK)
            bar += f"|  {k:&lt;11}{col}{'#'*filled}{'.'*(10-filled)}{C_RESET} {v:&gt;3}%{'':&gt;56}|\n"
        print(bar, end="")
        print("+" + "-" * 86 + "+")
        act = [d.name for d in self.bdi.D if d.status == DStatus.ACTIVE]
        ach = [d.name for d in self.bdi.D if d.status == DStatus.ACHIEVED]
        cur = self.bdi.I[-1] if self.bdi.I else None
        print(f"| DESIRES active: {', '.join(act) or '-':&lt;66}|")
```

```
    print(f"| DESIRES done  : {', '.join(ach) or '-':&lt;66}|")
    print(f"| INTENTION     : {(cur.plan + ' ' + cur.prog()) if cur else 'idle':&lt;66}|")
    print("+" + "-" * 86 + "+")
    print(f"| SECURITY  accepted={self.sec.ok:&lt;4} blocked={self.sec.blocked:&lt;4} "
          f"| PERCEPTS {self.percepts:&lt;4} | REFLEX FIRES {self.reflex_fires:&lt;4}"
          f"| BELIEF REVS {self.bdi.B.revisions:&lt;4}|")
    print("+" + "-" * 86 + "+")

# ---------------- FULL LIFECYCLE ---------------------------------
def run(self, cycles=8):
    print("=" * 88)
    print(" CAPSTONE :: AUTONOMOUS PLACEMENT PREPARATION AGENT (Assignment 5.2)")
    print(" Modules: U1 Reflex | U2 Daemons+Bus | U3 CNP+Utility | U4 BDI+KB | U5 Crypto")
    print("=" * 88)

    workers = [threading.Thread(target=f, daemon=True, name=n) for f, n in [
        (coding_daemon, "CodingDaemon"), (aptitude_daemon, "AptitudeDaemon"),
        (drive_daemon, "DriveDaemon"), (intruder_daemon, "IntruderDaemon")]]
    for w in workers:
        w.start()
        log(2, "DAEMON", f"started '{w.name}' (daemon={w.daemon})")

    # --- one-time strategic planning (Unit 3) ---
    print("\n" + "-" * 88)
    log(3, "OPTIMIZER", "running constrained utility maximization over 10 options")
    best, ranked = self.opt.optimize()
    if best:
        self.selected_resource = best
        for u, o in ranked[:4]:
            log(3, "OPTIMIZER", f"  {o[0]:&lt;16} U={u}")
        log(3, "OPTIMIZER", f"OPTIMAL -&gt; '{best[1][0]}' U={best[0]} "
                            f"(Rs.{best[1][1]:,}, {best[1][2]}h)", C_OK)
    print()
    self.contract = self.neg.run()

    # --- autonomous loop ---
    for c in range(1, cycles + 1):
        self.cycle = c
        print("\n" + "=" * 88)
        print(f" AUTONOMOUS CYCLE {c}/{cycles}")
        print("=" * 88)
        n = self.sense()                      # U1 sensors + U2 bus + U5 gate
        log(2, "BUS", f"drained {n} messages from event bus")
        self.reflex_pass()                    # U1
        self.bdi.deliberate()                 # U4
        for _ in range(2):
            self.bdi.execute()                # U4
        self.dashboard()                      # UI
        time.sleep(0.9)

    # --- shutdown ---
    print("\n" + "-" * 88)
    SHUTDOWN.set()
    for w in workers:
        w.join(timeout=2)
        log(2, "DAEMON", f"'{w.name}' joined (alive={w.is_alive()})")

    self.final_report()

def final_report(self):
    B = self.bdi.B
    print("\n" + "=" * 88)
    print(" FINAL SYSTEM REPORT")
    print("=" * 88)
    print(f" UNIT 1  reflex rules fired          : {self.reflex_fires}")
    print(f" UNIT 2  authenticated percepts      : {self.percepts} (4 daemon threads)")
    if self.contract:
        u, b = self.contract
        print(f" UNIT 3  negotiated contract         : {b['agent']} @ Rs.{b['fee']:,.0f}, "
              f"{b['slots']} slots, buyer-U={u}")
    if self.selected_resource:
        u, o = self.selected_resource
        print(f" UNIT 3  optimal resource selected   : {o[0]} (U={u})")
    print(f" UNIT 4  belief revisions            : {B.revisions}")
    print(f" UNIT 4  intentions dropped/replanned: {len(self.bdi.dropped)}")
    for i in self.bdi.dropped:
        print(f"            - {i.plan} dropped at {i.prog()}")
    print(f" UNIT 5  commands accepted / blocked : {self.sec.ok} / {self.sec.blocked}")
    for r in sorted(set(self.sec.blocked_events)):
        print(f"            - blocked: {r}")
```

```
        lv = B.get("levels")
        print(f"\n ELIGIBILITY (cgpa {B.get('cgpa')}):")
        for n, t, cov, lpa in self.kg.eligible(B.get("cgpa"), lv):
            print(f"    {n:&lt;12} tier-{t}  skill-coverage {cov*100:&gt;5.0f}%  {lpa} LPA")
        print(f"\n STATUS: agent completed {self.cycle} autonomous cycles with no human input.")
        print("=" * 88)
```

5. Result

The run completed 6 autonomous cycles with no human input. Unit 3 awarded the contract to Mentor-C at Rs 5,367 for 7 slots (buyer utility 0.6616). Unit 5 accepted 9 commands and blocked 2 (an untrusted host, and a malformed payload). By the final cycle the reflex layer had fired 14 times and belief was revised 7 times.

Execution output (log file)

Captured from **logs/5.2_capstone_agent.log** (python3 capstone_agent.py --cycles 6). Timestamps and some values differ between runs; this is the recorded run.

```
# Activity 5.2_capstone_agent
# command: python3 capstone_agent.py --cycles 6
# exit code: 0

============================================================================================
 CAPSTONE :: AUTONOMOUS PLACEMENT PREPARATION AGENT (Assignment 5.2)
 Modules: U1 Reflex | U2 Daemons+Bus | U3 CNP+Utility | U4 BDI+KB | U5 Crypto
============================================================================================
[12:26:33] [U2] [DAEMON     ] started 'CodingDaemon' (daemon=True)
[12:26:33] [U2] [DAEMON     ] started 'AptitudeDaemon' (daemon=True)
[12:26:33] [U2] [DAEMON     ] started 'DriveDaemon' (daemon=True)
[12:26:33] [U2] [DAEMON     ] started 'IntruderDaemon' (daemon=True)

--------------------------------------------------------------------------------------------
[12:26:33] [U3] [OPTIMIZER  ] running constrained utility maximization over 10 options
[12:26:33] [U3] [OPTIMIZER  ]   Product Sprint   U=0.6513
[12:26:33] [U3] [OPTIMIZER  ]   DSA Bootcamp     U=0.6394
[12:26:33] [U3] [OPTIMIZER  ]   Mock Bundle      U=0.6339
[12:26:33] [U3] [OPTIMIZER  ]   Core CS Kit      U=0.6058
[12:26:33] [U3] [OPTIMIZER  ] OPTIMAL -&gt; 'Product Sprint' U=0.6513 (Rs.9,900, 55h)

[12:26:33] [U3] [CNP        ] CFP broadcast: 6 mock slots under Rs.9,000
[12:26:33] [U3] [CNP        ]   R1 Mentor-C: Rs.5,321 7sl q8 -&gt; U=0.664
[12:26:33] [U3] [CNP        ]   R1 Mentor-A: Rs.5,720 5sl q9 -&gt; U=0.6457
[12:26:33] [U3] [CNP        ]   R1 Mentor-B: Rs.5,104 8sl q7 -&gt; U=0.6398
[12:26:33] [U3] [CNP        ]   R2 Mentor-C: Rs.5,302 7sl q8 -&gt; U=0.6649
[12:26:33] [U3] [CNP        ]   R2 Mentor-A: Rs.5,578 5sl q9 -&gt; U=0.6528
[12:26:33] [U3] [CNP        ]   R2 Mentor-B: Rs.5,168 8sl q7 -&gt; U=0.6366
[12:26:33] [U3] [CNP        ]   R3 Mentor-C: Rs.5,367 7sl q8 -&gt; U=0.6616
[12:26:33] [U3] [CNP        ]   R3 Mentor-A: Rs.5,573 5sl q9 -&gt; U=0.653
[12:26:33] [U3] [CNP        ]   R3 Mentor-B: Rs.5,345 8sl q7 -&gt; U=0.6277
[12:26:33] [U3] [CNP        ] ACCEPT Mentor-C U=0.6616 &gt;= 0.63
[12:26:33] [U3] [CNP        ] HANDSHAKE COMPLETE -&gt; contract awarded to Mentor-C @ Rs.5,367

============================================================================================
 AUTONOMOUS CYCLE 1/6
============================================================================================
[12:26:33] [U4] [BELIEF     ] rev#1 aptitude: 62 -&gt; 93 (aptitude-daemon)
[12:26:33] [U4] [BELIEF     ] rev#2 days_to_drive: 40 -&gt; 3 (placement-cell/Freshworks)
[12:26:33] [U4] [BELIEF     ] rev#3 announced_company: None -&gt; Freshworks (placement-cell)
[12:26:33] [U5] [SECURITY   ] BLOCKED: untrusted host 'evil-host'
[12:26:33] [U2] [BUS        ] drained 4 messages from event bus
[12:26:33] [U1] [REFLEX     ] R1_URGENT -&gt; switch to RAPID-REVISION (3d left)
[12:26:33] [U1] [REFLEX     ] R2_CODING -&gt; assign graded ladder on 'dp' (39%)
[12:26:33] [U1] [REFLEX     ] R5_IDLE -&gt; send reminder (0.6h studied today)
[12:26:33] [U4] [DESIRE     ] 'PASS_APTITUDE' ACHIEVED
[12:26:33] [U4] [INTENTION  ] COMMIT PLAN_EmergencyCram (desire SURVIVE_DRIVE, prio 30.0)
[12:26:33] [U4] [EXECUTE    ] [PLAN_EmergencyCram 1/5] freeze long-horizon goals
[12:26:33] [U4] [EXECUTE    ] [PLAN_EmergencyCram 2/5] load company question bank

+------------------------------------------------------------------------------------------+
| PLACEMENT PREPARATION AGENT :: LIVE DASHBOARD              cycle   1  12:26:33  |
+------------------------------------------------------------------------------------------+
| CGPA 7.6   Target: Freshworks   Drive in: 3d   Aptitude: 93%   Mock: 5/10    |
+------------------------------------------------------------------------------------------+
|  dp         ###.......  39%                                                              |
|  graphs     ####......  44%                                                              |
|  os         #####.....  55%                                                              |
|  recursion  #####.....  58%                                                              |
```

```
|  trees      ######....  63%                                                          |
|  dbms       ######....  68%                                                          |
|  oops       #######...  72%                                                          |
|  strings    #######...  76%                                                          |
|  arrays     ########..  81%                                                          |
|  aptitude   #########.  93%                                                          |
+--------------------------------------------------------------------------------------+
| DESIRES active: SURVIVE_DRIVE, FIX_WEAK_SKILL, INTERVIEW_READY                      |
| DESIRES done  : PASS_APTITUDE                                                       |
| INTENTION     : PLAN_EmergencyCram 2/5                                              |
+--------------------------------------------------------------------------------------+
| SECURITY  accepted=3    blocked=1    | PERCEPTS 3    | REFLEX FIRES 3   | BELIEF REVS 3   |
+--------------------------------------------------------------------------------------+

======================================================================================
 AUTONOMOUS CYCLE 2/6
======================================================================================
[12:26:34] [U2] [BUS        ] drained 0 messages from event bus
[12:26:34] [U1] [REFLEX     ] R1_URGENT -&gt; switch to RAPID-REVISION (3d left)
[12:26:34] [U1] [REFLEX     ] R2_CODING -&gt; assign graded ladder on 'dp' (39%)
[12:26:34] [U4] [EXECUTE    ] [PLAN_EmergencyCram 3/5] revise top-50 problems

... (middle of run omitted; full log in logs/5.2_capstone_agent.log) ...

 UNIT 4  intentions dropped/replanned: 0
 UNIT 5  commands accepted / blocked : 9 / 2
            - blocked: malformed payload
            - blocked: untrusted host 'evil-host'

 ELIGIBILITY (cgpa 7.6):
    Zoho         tier-2  skill-coverage    67%  18 LPA
    Freshworks   tier-2  skill-coverage    33%  22 LPA
    Infosys      tier-3  skill-coverage   100%  6.5 LPA

 STATUS: agent completed 6 autonomous cycles with no human input.
======================================================================================
```

6. Viva points

What is the single agent here? PlacementPreparationAgent; the daemons, mentors and intruder are its sensors, peers and adversary.

The most important demonstration is that the blocked messages do not change the agent's state.

Output includes ANSI colour codes in the terminal; the log file has them removed.

7. Conclusion

All five units work together in one running loop, and the security layer holds while the agent keeps working.